

A Feasibility-Aware HUBO/QUBO Benchmark for UAV Obstacle-Avoidance with Visibility Cost

Author names to be finalized

Affiliations to be finalized

Email: to be finalized

Abstract—Quantum-assisted UAV path planning has recently been studied using QAOA-style obstacle-avoidance formulations, but a low-energy binary sample is not necessarily a valid UAV trajectory. This paper presents a feasibility-aware benchmark for a discrete 8×8 , $L = 20$ UAV grid-planning problem with obstacle avoidance, a line-of-sight visibility cost, and a higher-order temporal buffer-risk term. The horizon L denotes discrete planning layers, not physical seconds. The model uses binary variables $x_{t,v}$ indicating whether the UAV occupies cell v at time layer t and forms a HUBO objective with one-hot, start, motion, obstacle, visibility, terminal-goal, and sustained near-obstacle proximity terms. The instance has 1,280 original binary variables and 118,344 polynomial terms, including 4,392 cubic monomials. We compare A*, RRT-best, a QAOA-style QAOA candidate-path selector, native trajectory-level simulated annealing (SA), D-Wave Ocean’s classical `neal` sampler after HUBO-to-QUBO reduction, and an exact CP-SAT/MILP baseline that minimizes the native HUBO soft objective over hard-feasible paths. The CP-SAT solver returns OPTIMAL and certifies that the best observed SA path is globally optimal for the enforced feasible-path objective, with native HUBO energy -20752.43 , soft cost 247.57, path length 14, six turns, and 85% visibility. We also include an IBM/Qiskit-compatible Hamiltonian mapping to show how the binary polynomial model can be represented as a Pauli- Z cost Hamiltonian on reduced instances. The full instance is not claimed to be executable on present gate-based hardware because a direct mapping would require 1,280 qubits. The results do not claim quantum speedup; they show that feasibility-aware decoding, exact baselines, quadratization accounting, and Hamiltonian compatibility are essential for interpreting quantum-compatible UAV planning results.

Index Terms—HUBO, QUBO, feasibility-aware benchmarking, UAV path planning, obstacle avoidance, visibility-aware planning, MILP, CP-SAT, QAOA, quantum-compatible optimization.

I. INTRODUCTION

UAV path planning in cluttered environments requires at least two safety properties: the path must avoid obstacles, and the decoded plan must be executable under the allowed motion model. When perception or tracking is relevant, the planner may also prefer cells with line of sight to the target. Recent quantum-assisted work such as QAOA formulates UAV obstacle-avoidance path planning using QAOA-style optimization and compares against A* and RRT. QAOA is therefore an important closest reference for quantum-assisted UAV navigation. However, a QAOA, annealing, or QUBO solver returns a binary sample, and the sample must be decoded and checked before it can be called a valid path.

This paper studies a deliberately compact grid abstraction so that modeling and solver behavior can be inspected directly.

The goal is not to claim quantum speedup on a small static grid. The goal is to define a reproducible HUBO/QUBO-compatible benchmark, separate energy minimization from actual path validity, and compare classical, quantum-inspired, and quantum-compatible solver pathways on the same problem instance.

A. Problem statement and scope

The planning problem considered here is a static, discrete visibility-aware path-planning abstraction. A UAV must move from a fixed start cell to a fixed target/goal cell through legal grid transitions while avoiding obstacle cells. Visibility is represented as a soft line-of-sight cost from each candidate UAV cell to the fixed target cell. Thus, this paper does not solve continuous quadrotor control, full moving-target tracking, or camera field-of-view control. The abstraction is intentionally small enough that the encoded binary model, decoded paths, feasibility failures, and exact classical baselines can be inspected in detail. The time horizon L is a design parameter giving the number of discrete planning layers. It is not assumed to be seconds unless a physical sampling time Δt is separately assigned.

The contributions are as follows. First, we give a complete binary decision-variable formulation for UAV grid planning with obstacle avoidance and a visibility cost. Second, we identify the real source of higher-order structure: a temporal buffer-risk term, not the static line-of-sight cost. Third, we introduce a feasibility-aware evaluation protocol that reports structural feasibility, goal arrival, and full-task success separately. Fourth, we implement a same-platform QAOA-style QAOA candidate-path baseline, clearly not as official QAOA code but as a reproducible reference following the published workflow. Fifth, we add exact CP-SAT and MILP implementations, enabling global comparison against the native HUBO soft objective over hard-feasible paths. Sixth, we provide an IBM/Qiskit-compatible Hamiltonian mapping for reduced gate-based QAOA experiments.

The novelty should be interpreted narrowly. This paper does not introduce a faster planner than A* or CP-SAT, and it does not demonstrate quantum advantage. Its novelty is the feasibility-aware benchmarking layer around a quantum-compatible UAV HUBO/QUBO formulation: the same decoded paths are compared under graph-search, candidate-path QAOA, native trajectory-SA, quadratized BQM sampling, and exact CP-SAT/MILP baselines, with validity and goal arrival

separated from energy. This positioning is intentional because, without exact baselines and decoded-path checks, low-energy quantum-compatible samples can be misleading.

II. DISCRETE PLANNING MODEL

Let $G = (V, E)$ be an $N \times N$ grid graph. In the experiments $N = 8$, $|V| = 64$, and time layers are indexed $t = 0, \dots, L-1$ with $L = 20$. Each time layer is one discrete trajectory frame; a larger L gives the planner more room for detours or hovering but increases the number of variables linearly. Let $O \subset V$ be the obstacle set, $s \in V$ the start cell, and $g \in V$ the target cell. The permitted next-cell set is

$$\mathcal{N}(u) = \{u\} \cup \{v \in V : (u, v) \in E\}, \quad (1)$$

where the self-loop represents hovering.

The binary state variable is

$$x_{t,v} \in \{0, 1\}, \quad x_{t,v} = 1 \Leftrightarrow \text{the UAV is at cell } v \text{ at time } t. \quad (2)$$

A valid binary solution decodes to a trajectory $\pi = (v_0, \dots, v_{L-1})$ only if exactly one state variable is active at every layer.

Let $\text{LOS}(u, g)$ denote the rasterized Bresenham line cells strictly between u and g . The per-cell visibility/occlusion-depth surrogate is

$$C_{\text{occ}}(u) = |\text{LOS}(u, g) \cap O|. \quad (3)$$

A zero value indicates clear line of sight in this grid model. Let $B \subset V \setminus O$ be the set of non-obstacle cells within Chebyshev distance one of an obstacle. Cells in B are legal but risky.

III. HUBO FORMULATION

The total polynomial objective is

$$H(x) = H_{\text{uniq}} + H_{\text{start}} + H_{\text{move}} + H_{\text{obs}} + H_{\text{occ}} + H_{\text{prox}} + H_{\text{goal}} + H_{\text{term}}. \quad (4)$$

The structural terms are

$$H_{\text{uniq}} = \lambda_u \sum_{t=0}^{L-1} \left(\sum_{v \in V} x_{t,v} - 1 \right)^2, \quad (5)$$

$$H_{\text{start}} = \lambda_s (1 - x_{0,s}), \quad (6)$$

$$H_{\text{move}} = \lambda_m \sum_{t=0}^{L-2} \sum_{u \in V} \sum_{v \notin \mathcal{N}(u)} x_{t,u} x_{t+1,v}, \quad (7)$$

$$H_{\text{obs}} = \lambda_o \sum_{t=0}^{L-1} \sum_{v \in O} x_{t,v}. \quad (8)$$

The static visibility cost is linear:

$$H_{\text{occ}} = \lambda_{\text{occ}} \sum_{t=0}^{L-1} \sum_{u \in V} C_{\text{occ}}(u) x_{t,u}. \quad (9)$$

The sole cubic term is the sustained-proximity penalty:

$$H_{\text{prox}} = \lambda_p \eta \sum_{t=1}^{L-2} \sum_{u \in B} \sum_{v \in B \cap \mathcal{N}(u)} \sum_{w \in B \cap \mathcal{N}(v)} x_{t-1,u} x_{t,v} x_{t+1,w}. \quad (10)$$

Here $\eta = 3$. This term penalizes remaining in connected buffer cells for three consecutive layers. Thus the model is genuinely HUBO, but the higher-order degree comes from temporal buffer-risk modeling rather than from static visibility.

The goal terms are

$$H_{\text{goal}} = \lambda_g \sum_{t=0}^{L-1} \sum_{v \in V} \frac{t+1}{L} \|p_v - p_g\|_2 x_{t,v}, \quad (11)$$

$$H_{\text{term}} = \lambda_{\text{term}} \sum_{v \neq g} x_{L-1,v}. \quad (12)$$

The updated instance uses $\lambda_u = \lambda_s = \lambda_m = \lambda_o = 1000$, $\lambda_{\text{occ}} = 10$, $\lambda_p = 5$, $\lambda_g = 8$, $\lambda_{\text{term}} = 50$, and $\eta = 3$.

IV. FEASIBILITY-AWARE EVALUATION

A solver output is not scored only by total HUBO energy. Each binary sample is decoded and checked directly. We report: structural feasibility, meaning one-hot occupancy, correct start, legal transitions, and no obstacle occupancy; goal arrival, meaning $v_{L-1} = g$; and full-task success, meaning both conditions hold for the same decoded sample. Soft cost and native HUBO energy are then interpreted on valid decoded paths.

This distinction is essential for QUBO/BQM and QAOA-style workflows. A sample can have a low reduced-model energy while failing one-hot, motion, obstacle, or target-arrival checks. Such a sample is not a UAV plan. The benchmark therefore reports feasibility rates and representative decoded trajectories, not energy alone.

V. SOLVERS AND IMPLEMENTATION

A. A* and RRT Baselines

A* searches directly on the grid with obstacle avoidance and a heuristic path cost. RRT-best is a randomized grid planner used as a sampling-style reference. These methods return explicit paths and are checked with the same feasibility evaluator.

B. QUAV-Style QAOA Baseline

QUAV maps UAV path segments to qubits and applies QAOA-style optimization with obstacle-buffer costs. Since no official public code was available in this workflow, we implemented a QUAV-style candidate-path selector following the published algorithmic description. It generates collision-free candidate paths, assigns a QUAV-style geometric cost using length, buffer steps, and turns, and uses a small QAOA-style one-hot selector over the candidates. This baseline is a same-platform reference only, not an exact reproduction of the authors' implementation.

C. IBM/Qiskit Hamiltonian Compatibility

The native HUBO can also be written as a diagonal gate-based cost Hamiltonian. Renumber the binary variables as $x_i \in \{0, 1\}$ and use the standard mapping

$$x_i = \frac{\mathbb{I} - Z_i}{2}, \quad (13)$$

where Z_i is the Pauli-Z operator on qubit i . A general HUBO monomial maps as

$$c \prod_{i \in S} x_i \mapsto \frac{c}{2^{|S|}} \sum_{R \subseteq S} (-1)^{|R|} \prod_{i \in R} Z_i. \quad (14)$$

Therefore a cubic proximity term expands into a sum of constant, one-body, two-body, and three-body Z strings. The full cost Hamiltonian has the form

$$\hat{H}_C = c_0 + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j + \sum_{i < j < k} J_{ijk} Z_i Z_j Z_k. \quad (15)$$

This mapping makes the formulation compatible with Qiskit-style QAOA on reduced instances. It does not make the full 8×8 , $L = 20$ case immediately executable: a direct state-variable mapping would require $|V|L = 1,280$ qubits before any auxiliary variables. The accompanying code therefore includes a general HUBO/QUBO-to-Pauli expansion that can generate a `SparsePauliOp`-compatible representation. We treat this mapping as a mathematical and software compatibility path, not as a completed large-scale IBM hardware result.

D. Trajectory-Level SA and `neal`

Trajectory-level SA searches over cell sequences and evaluates the native HUBO energy directly, including the cubic term. Its proposal operator preserves one-hot occupancy, start, legal motion, and obstacle avoidance by construction. In contrast, D-Wave Ocean’s `neal` is a classical BQM sampler. Therefore the HUBO is first reduced to a QUBO/BQM with auxiliary variables and quadratization penalties before sampling.

E. Exact MILP and CP-SAT Baselines

We added exact optimization baselines in both CP-SAT and MILP form. Both encodings enforce the path constraints directly: exactly one cell per layer, fixed start, final target arrival, obstacle cells forbidden, and legal transitions. Auxiliary binary variables linearize each cubic proximity monomial $y_{tuvw} = x_{t-1,u} x_{t,v} x_{t+1,w}$. The objective minimizes

$$H_{\text{occ}} + H_{\text{prox}} + H_{\text{goal}} + H_{\text{term}} \quad (16)$$

over the hard-feasible set. Because all hard constraints are enforced, every feasible solution has the same hard contribution $-\lambda_u L - \lambda_s = -21000$ under the expanded HUBO implementation. The native HUBO energy is therefore this hard constant plus the optimized soft cost. The CP-SAT model uses OR-Tools and the MILP model uses a linear mixed-integer encoding. The CP-SAT run returned OPTIMAL, so its solution is used as the exact baseline in the final table.

VI. RESULTS

Table I summarizes the updated instance.

TABLE I: Updated main HUBO instance.

Quantity	Value
Grid size	8×8
Horizon L	20
Original binary variables	1,280
Total polynomial terms	118,344
Linear terms	1,280
Quadratic terms	112,672
Cubic terms	4,392
Obstacles	6
Buffer cells	24

TABLE II: Unified same-platform comparison. For deterministic or single-output methods, feasibility columns indicate whether the returned path satisfies the condition. For stochastic solvers, feasibility columns report empirical rates over 50 runs/reads.

Method	Feas.	Goal	Full	Len.	Turns	Buf.	Vis.	HUBO E	Time
A*	Yes	Yes	Yes	14	7	4	85.0%	-20742.26	0.028s
RRT	Yes	Yes	Yes	14	5	4	70.0%	-20700.19	N/R
QUAV-QAOA	Yes	Yes	Yes	14	1	3	80.0%	-20725.94	4.11s
CP-SAT exact	Yes	Yes	Yes	14	6	4	85.0%	-20752.43	1.02s
HUBO-SA	100.0%	16.0%	16.0%	14	6	4	85.0%	-20752.43	1207.26s
QUBO/ <code>neal</code>	12.0%	8.0%	0.0%	18	12	2	20.0%	-20060.47	6.15s

A. Decoded Optimized Paths

Fig. 1 shows the actual decoded paths returned by each method on the same map. This figure is important because the numerical table alone does not reveal how the methods use the obstacle-buffer region or why different objectives prefer different routes. A*, RRT-best, QUAV-style QAOA, CP-SAT, and the best full-success SA trajectory all reach the target without collisions. The `neal` panel shows the best-energy decoded QUBO sample; it is structurally feasible but fails to reach the target, explaining why its full-task success rate is zero.

The exact CP-SAT baseline reports OPTIMAL, with soft cost 247.57 and native HUBO energy -20752.43 in 1.02 s. This matches the best trajectory-SA energy, showing that SA found a globally optimal path for the hard-feasible exact objective on this instance, but SA required 1207.26 s over 50 runs and reached the target in only 8 of 50 runs. A* is much faster and fully successful, but its native HUBO energy is slightly worse because it optimizes a graph-search surrogate rather than the exact cubic proximity-aware objective.

The QUAV-style QAOA selector produces a valid shortest path with only one turn and the lowest QUAV-style geometric cost among its candidates. However, under the native HUBO objective its energy is -20725.94 , worse than the CP-SAT/SA optimum. This illustrates a central point: a QUAV-style geometric objective and the native HUBO objective are not the same, even when both return collision-free paths. `neal` after quadratization remains the weakest method in this run: it has 6/50 structural feasibility and 4/50 goal arrival, but 0/50 full-task success.

B. Penalty Sweep

The penalty-weight sweep for `neal` shows that weak structural penalties permit low-energy but infeasible samples.

Decoded optimized paths on the common 8x8 UAV benchmark

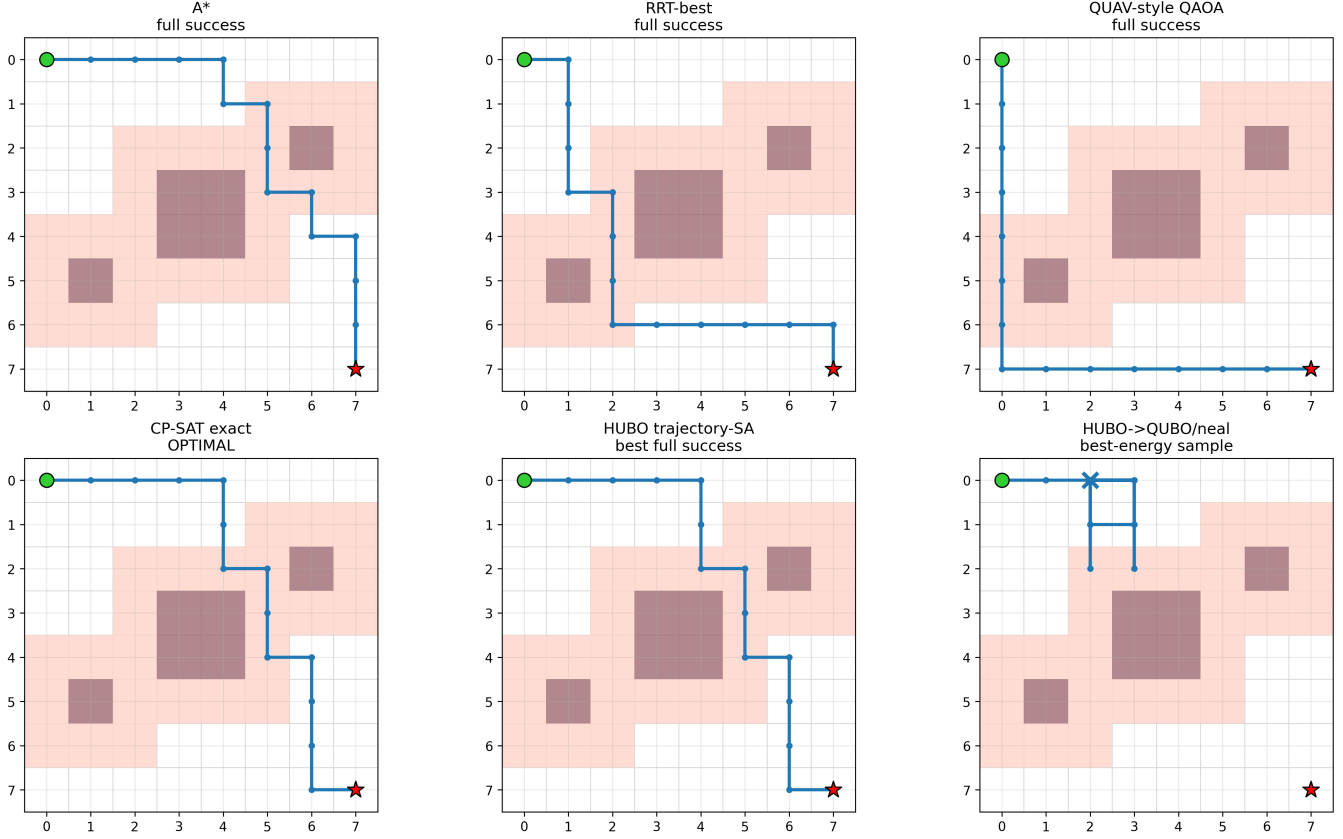


Fig. 1: Decoded optimized paths on the common 8×8 , $L = 20$ benchmark. Dark cells are obstacles, light cells are obstacle-buffer cells, the green circle is the start, and the red star is the target. The CP-SAT path is globally optimal for the hard-feasible native-HUBO soft objective. The *neal* path is the best-energy decoded QUBO sample and illustrates a quadratization/sampling failure mode: it is structurally feasible but does not reach the target.

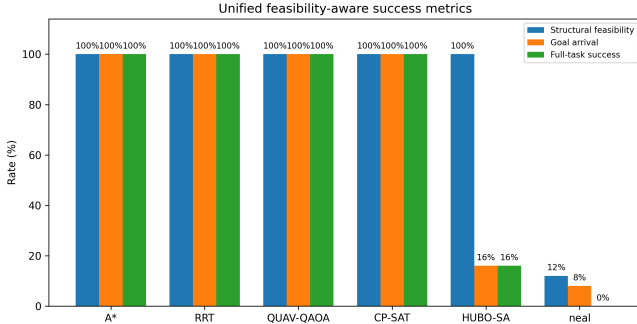


Fig. 2: Unified feasibility-aware success metrics. The exact CP-SAT run certifies a full-success optimum. *neal* has nonzero feasibility and goal-arrival rates separately, but zero full-task success.

In the updated run, structural feasibility was 0/15 for $\lambda \in \{50, 100, 300, 600\}$, 4/15 at $\lambda = 1000$, and 5/15 at $\lambda = 2000$. The best feasible soft cost improved from 915.4 to 590.6 between $\lambda = 1000$ and $\lambda = 2000$, still worse than A*, SA, and CP-SAT.

VII. DISCUSSION

The results show why the paper’s contribution is not a simple claim that quantum computing outperforms classical planning. On this instance, exact CP-SAT proves the optimum of the hard-feasible native-HUBO soft objective, A* is very fast, and trajectory-SA can find the exact optimum but with poor target-arrival reliability. The decoded paths in Fig. 1 make the solver differences explicit. The QUAU-style QAOA baseline is useful because it shows that a quantum-assisted geometric selector can produce a valid obstacle-free path on the same platform. However, the native HUBO objective distinguishes visibility, temporal buffer-risk, and terminal behavior in a different way. Therefore, same-platform decoded-path evaluation is necessary for fair comparison.

The exact CP-SAT/MILP formulation also clarifies the role of quantum-compatible solvers. If a QUBO sampler is used, it must overcome not only path planning but also one-hot, movement, obstacle, target, and quadratization constraints. The benchmark exposes these failure modes by reporting structural feasibility, goal arrival, and full-task success separately.

The IBM/Qiskit and D-Wave routes have different math-

ematical roles. For IBM/Qiskit, the HUBO must be represented as a Pauli cost Hamiltonian, or the model must be quadratized to a QUBO and then converted to an Ising Hamiltonian. The Pauli mapping confirms mathematical compatibility, but it does not by itself demonstrate scalable QAOA performance. The QUAV-style row in Table II should also be interpreted carefully: it is a candidate-path selector over generated paths, not a native solution of the full $x_{t,v}$ HUBO. This candidate-path route is practical because feasibility is mostly handled before quantum optimization, but it optimizes over a restricted candidate set. Thus the quantum-compatible components demonstrate formulation and benchmarking pathways, not superiority over A*, CP-SAT, or MILP. Those classical methods remain the correctness anchors. For D-Wave BQM/QPU workflows, the cubic term must also be reduced to quadratic form with auxiliary variables; D-Wave CQM can express constraints more naturally, but the cubic term still requires auxiliary modeling. Consequently, the present paper treats D-Wave hardware/hybrid execution and larger IBM/Qiskit studies as future experimental paths rather than as completed performance claims.

VIII. LIMITATIONS AND FUTURE WORK

The model remains a grid abstraction without quadrotor dynamics, velocity, acceleration, heading, camera field-of-view dynamics, or continuous controls. The target is static, so the visibility term is linear; a moving-target or occlusion-free tracking formulation would require a separate derivation with time-varying target states and hard line-of-sight constraints. The QUAV-style baseline is not official QUAV code. The IBM/Qiskit contribution is currently a Hamiltonian-compatibility mapping rather than a completed hardware or scalable QAOA experiment. D-Wave QPU/hybrid execution, larger Qiskit QAOA tests, moving-target formulations, and larger maps are natural future work.

IX. CONCLUSION

This paper presents a feasibility-aware HUBO/QUBO benchmark and solver-analysis framework for UAV obstacle-avoidance with visibility cost. The higher-order structure comes from sustained temporal proximity to obstacle-buffer cells. The unified comparison with A*, RRT-best, QUAV-style QAOA, trajectory-SA, `neal`, and exact CP-SAT shows that geometric path quality, native HUBO energy, and full trajectory validity are different metrics. The CP-SAT run certifies the best observed SA path as optimal for the hard-feasible native-HUBO soft objective, while `neal` after quadratization has zero full-task success in the tested reads. The added Pauli-Z mapping clarifies how the HUBO/QUBO can be represented for IBM/Qiskit-style experiments. The main contribution is therefore a rigorous quantum-compatible benchmark and feasibility-aware evaluation protocol, not a quantum-speedup claim.

CODE AVAILABILITY

The code package for this revision includes the HUBO builder, A* baseline, RRT/QUAV-style baseline, trajectory-SA,

`neal` benchmark, exact CP-SAT baseline, companion MILP model, HUBO-to-Pauli Qiskit mapping script, plotting scripts, and result logs. A public repository or supplementary-material link should be inserted before submission.

REFERENCES

- [1] N. Innan, M. Kashif, A. Marchisio, Y.-S. Gan, F. Barbaresco, and M. Shafique, “QUAV: Quantum-assisted path planning and optimization for UAV navigation with obstacle avoidance,” arXiv:2508.21361, 2025.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” Tech. Rep. 9811, Iowa State University, 1998.
- [4] I. G. Rosenberg, “Reduction of bivalent maximization to the quadratic case,” *Cahiers du Centre d’Etudes de Recherche Operationnelle*, vol. 17, pp. 71–74, 1975.
- [5] F. Glover, G. Kochenberger, and Y. Du, “Quantum bridge analytics I: A tutorial on formulating and using QUBO models,” *4OR*, vol. 17, pp. 335–371, 2019.
- [6] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” arXiv:1411.4028, 2014.
- [7] D-Wave Systems, “dwave-neal simulated annealing sampler,” software package.
- [8] D-Wave Systems, “dimod: A shared API for binary quadratic models,” software package.
- [9] Qiskit contributors, “Qiskit, Qiskit Optimization, Qiskit Algorithms, and Qiskit Aer,” software packages.
- [10] P. Virtanen et al., “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [11] L. Perron and V. Furnon, “OR-Tools,” Google, software package.